

Corrigible Structural Interfaces

Evidence-Bound Feedback, Provenance-Preserving Revision, and Revalidation

WuJun Chen
Independent Researcher
2026

This paper is Paper XV of the RIME program. It closes the numbered SOF protocol line by defining evidence-bound revision, supersession, and revalidation of current commitments. Action selection, authorization, execution, repair, outcome, and effect remain external.

Abstract

Problem. A trustworthy structural interface must preserve prior evidence while allowing later admissible evidence to revise its current use. Without a typed revision boundary, correction risks either silent mutation or an unqualified newest-evidence-wins rule.

Approach. This paper separates object state X_t from epistemic state E_t and defines five interface objects: `EpistemicState`, `EvidenceUpdate`, `RevisionRecord`, `SupersessionMap`, and `RevalidationObligation`. Admitted evidence and an active, versioned revision policy produce a non-authoritative proposal with explicit supersession and revalidation records. The proposal becomes current only after conformance validation and a separate activation basis. Earlier artifact bytes and their declared historical semantic bindings remain immutable.

Results. Five interface propositions establish that provenance history is monotone while current-use bindings may be nonmonotone; artifact identity is distinct from the commitment entry that gives it a role, scope, and validation basis; supersession forms new entries and explicit semantic edges; affected dependents do not inherit current validity without revalidation; and feedback does not authorize object intervention. An explicit two-state construction also shows that typed, auditable revision need not converge.

Boundary. The construction is not a control, planning, causal-identification, governance-rights, or convergence theorem. Recency alone does not establish truth, audit mismatch does not require repair, and local correction does not imply convergence to truth or a global optimum.

1 Introduction

The SOF program now separates a static realization, an observation along a separately supplied object trajectory, capability-aware compilation, wall morphology, a single report, an aligned audit, and context-relative bounded candidates [6, 5, 4, 10, 8, 9, 7]. This separation establishes a causal firewall:

object transition $\neq$ observation record $\neq$ report,
aligned difference $\neq$ interpretation $\neq$ candidate,
candidate $\neq$ authorized execution.

While this firewall blocks one class of errors, it exposes another. Once a report, alignment, profile, policy, or interpretation has been validated, how can it later be corrected without pretending that the earlier artifact never existed? A longitudinal system needs to distinguish at least three statements:

1. an immutable artifact was valid under its original declared closure;
2. that artifact is still current under the present epistemic state;

3. the underlying object has or has not changed.

These statements are independent. A source object may remain fixed while an alignment is corrected and the resulting audit changes. Conversely, an object may change while the reporting configuration remains fixed until a new observation arrives. A previously valid artifact can remain historically valid under its old closure while no longer being current for a new decision.

This paper positions correction as a normal typed transition rather than an exception handler. Its central invariant is: **current semantics may evolve; historical semantic binding is immutable.**

Its central type distinction is

$$X_t \neq E_t.$$

1.1 Contributions

This paper makes seven contributions.

1. It defines `EpistemicState`, `EvidenceUpdate`, `RevisionRecord`, `SupersessionMap`, and `RevalidationObligation` as the five core objects.
2. It separates append-only provenance history, immutable artifact identity, and defeasible current-use commitment bindings.
3. It formalizes split/merge supersession without mutation of historical bytes or retroactive rebinding of declared semantic closure.
4. It distinguishes epistemic correction from object repair and intervention.
5. It establishes Feedback Non-Authority and a generic contestability boundary.
6. It formulates shadow revision and receding-horizon verifiability without claiming global optimization.
7. It gives an explicit oscillatory revision construction showing that type safety and auditability do not imply stabilization.

1.2 Scope

X_t belongs to the object layer. E_t belongs to an informational layer of evidence, configuration, and current commitments. This paper owns only the transition $E_t \rightarrow E_{t+1}$. Any $X_t \rightarrow X_{t+1}$ transition must be supplied by external dynamics or a separately authorized intervention.

The motivating operational analogy is finite-rationality planning: act or revise from the evidence presently available, preserve the reasons for the step, observe what follows, and permit later correction. This paper abstracts that pattern only. It does not encode a historical tactic, a military doctrine, or a universal strategy formula.

Acyclic Epistemic Revision

Object change remains external; currentness requires validation and activation

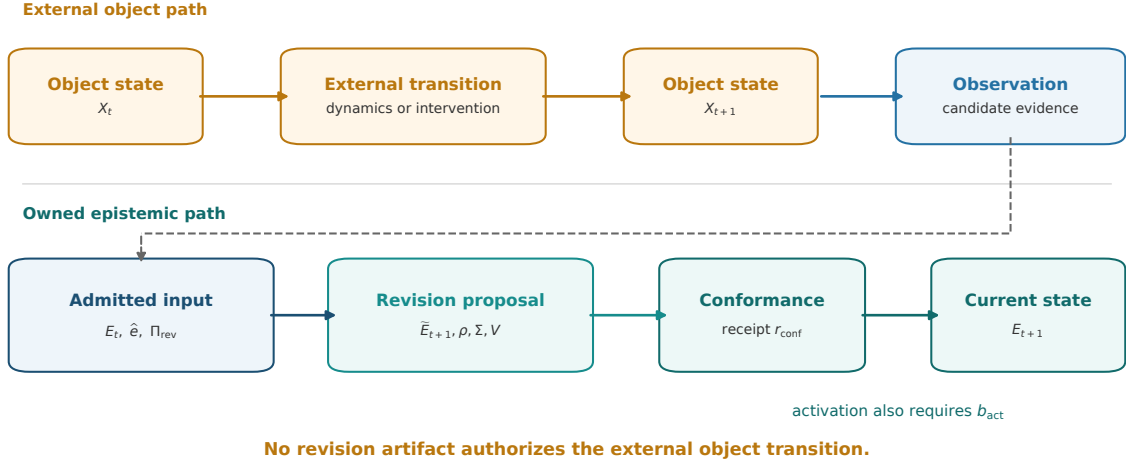


Figure 1. The external object path and the owned epistemic path remain distinct. Observation may supply candidate evidence, but revision begins only after admission. A proposal precedes its conformance receipt and becomes current only with a separate activation basis.

Notation Table

Symbol or term	Meaning
X_t	object or source state at index t
E_t	current SOF epistemic state; not an object state
H_t	append-only provenance history in E_t
K_t	finite set of current, defeasible typed commitment entries in E_t
Γ_t	active source-addressed realization, alignment, profile, policy, and contract references
c	commitment entry (role, subject_ref, scope, validation_basis_ref)
e	candidate EvidenceUpdate before admission
$\hat{e}$	admitted, scope-compatible evidence update
$\Pi_{adm,t}$	active versioned evidence-admission policy referenced by Γ_t
$\Pi_{rev,t}$	active versioned revision policy referenced by Γ_t
r_{adm}	source-addressed admission receipt for $e \mapsto \hat{e}$
b_{act}	explicit basis for making a conforming epistemic state current
r_{conf}	revision-protocol conformance receipt; not a currentness basis
$\rho_{t \rightarrow t+1}$	proposed RevisionRecord ; it contains no validating receipt or final-state reference
$\mathcal{L}_{t \rightarrow t+1}$	completed epistemic-revision lifecycle: proposal, conformance validation, and activation
$\Sigma_{t \rightarrow t+1}$	finite typed supersession relation from old to successor commitment entries or withdrawal markers
$V_{t \rightarrow t+1}$	RevalidationObligation for affected dependent commitments
$\text{Dep}_t^H(c)$	declared immutable history premises of commitment c
$\text{Dep}_t^K(c)$	declared current-use commitment dependencies of commitment c
CURRENT	eligible to be treated as current under the declared epistemic state
SUPERSEDED	retained in history but no longer current under the new state
REVALIDATION_REQUIRED	not inherited as current-valid after a dependency change

The index t orders epistemic versions. It need not be physical time. An epistemic state is a finite typed interface, not a deductively closed theory and not an estimate that is assumed to equal the world.

2 Related Work and Novelty Boundary

2.1 Belief Revision and Truth Maintenance

AGM belief revision studies rational postulates for contraction and revision of belief sets [1]. Truth-maintenance systems record dependencies and reasons so that assumptions can be withdrawn when conflicts appear [11]. Although this paper shares the premise that current commitments may be revised and that dependency information matters, it does not define a deductively closed belief set, adopt the AGM postulates as SOF axioms, or provide a general-purpose reasoner. Its narrower object is a typed, source-addressed revision of report-stack commitments.

2.2 Provenance and Versioned Data

Data-provenance research distinguishes where information came from and why a derived result exists [3, 14]. Schema-evolution work likewise separates persistent data from changing representations [2]. This paper uses provenance as an append-only historical layer, then adds a separate current-commitment layer, explicit supersession edges, and a revalidation obligation. It does not claim a new general provenance algebra or database migration algorithm.

2.3 Reversible Runtime Composition

Runtime composability work such as Cordis treats correction operationally: effects carry witnessed left inverses and declared dependencies react to changes in runtime context [13]. Under additional independence, totality, quiescence, and fixed-input conditions, its lifecycle calculus admits conditional confluence up to declared observational equivalences and fiber renaming. This paper addresses a different layer: rather than restoring an operational context or erasing runtime history, it forms successor epistemic bindings while preserving the admitted revision history. Thus a runtime inverse is not epistemic supersession, operational recovery is not historical erasure, and conditional runtime confluence does not imply epistemic convergence.

SOF adopts a source-addressed and version-bound admission discipline rather than key identity alone, while structural and behavioral compatibility remain separately validated obligations. In particular,

$$\begin{aligned} \text{identity/version integrity} &\neq \text{structural compatibility}, \\ \text{behavioral compatibility} &\neq \text{semantic adequacy}. \end{aligned}$$

2.4 Receding-Horizon Control

Model predictive control repeatedly solves a finite-horizon problem from the current plant state and applies only an initial control before replanning [12]. The analogy motivates the phrase receding-horizon verifiability, but the mathematical ownership is different. This paper does not optimize a control objective, model plant dynamics, apply a control input, or prove stability. Its horizon concerns the next epistemic transition and the evidence needed to audit it.

2.5 SOF Boundary

Paper IX separates an underlying object trajectory from its deformation record. Paper XII keeps compilation and report validation in the artifact layer. Paper XIII localizes aligned differences without causal attribution. Paper XIV stops at context- and policy-relative bounded candidates [5, 8, 9, 7]. This paper consumes immutable artifacts and validation references from those layers. It does not revise their theorem ownership or import execution semantics into them.

Novelty boundary. The contribution is the typed epistemic transition between one current source-addressed closure and the next: append-only history, nonmonotone commitments,

explicit supersession, and revalidation. It is not a new logic of belief change, a planner, a controller, or a causal model.

3 Object and Epistemic Layers

3.1 Object-State Transition

Let Obj be the application-owned object-state space. A transition

$$D_t : X_t \longrightarrow X_{t+1}$$

may be supplied by native dynamics, an environmental process, a parameter update, or a separately authorized external intervention. SOF does not infer D_t from a report difference. This paper does not define D_t .

Observation remains separately typed:

$$X_t \xrightarrow{\text{Observe}_{\eta,t}} \mathcal{F}_t \xrightarrow{\text{Compile/Assemble}} \mathcal{R}_t.$$

The report $\mathcal{R}_t$ is evidence about the declared observation. It is not a state-transition operator on Obj .

3.2 Epistemic State

Definition 3.1 (Epistemic State).

A finite SOF epistemic state is

$$E_t = (H_t, K_t, \Gamma_t),$$

where H_t is an append-only history of immutable evidence, artifact, commitment-entry, validation, and revision references; K_t is the finite set of typed commitment entries treated as current under this state; and Γ_t is the active source-addressed configuration of realization, alignment, admission policy, revision policy, report profile, application policy, and contract references.

A commitment entry has the typed form

$$c = (\text{role}, \text{subject_ref}, \text{scope}, \text{validation_basis_ref}).$$

The `subject_ref` points to an immutable report, audit, alignment, profile, policy, evidence object, interpretation, candidate disposition, or another source-addressed subject. Currentness belongs to the entry c , not to the bytes addressed by `subject_ref`. Two entries may therefore reference the same immutable subject while carrying different scope or validation bases. A revalidation may remove an old entry from K_t and add a successor entry that still points to the same subject.

artifact identity $\neq$ current-use binding.

This entry is binding metadata inside `EpistemicState`, not a sixth core artifact. Membership in K_t means current under the declared state, not universally true. An exact theorem remains exact under its hypotheses; what may be revised is the application claim that those hypotheses, inputs, or mappings are the current ones. Write

$$\text{Subjects}(K_t) = \{\text{subject_ref}(c) : c \in K_t\}.$$

The active configuration references in Γ_t require current-use bindings, so

$$\text{Refs}(\Gamma_t) \subseteq \text{Subjects}(K_t).$$

Γ_t types their active roles; the corresponding entries in K_t record their current-use bindings. In particular, $\Pi_{\text{adm},t}$ and $\Pi_{\text{rev},t}$ must both have active references in Γ_t and current entries in K_t . Each artifact in H_t remains bound to the declared contract, profile, policy, and validation closure under which it was formed. Although a later ontology or interpretation may supersede that historical semantic binding, it cannot retroactively rebind the earlier bytes to successor semantics.

3.3 Evidence Update

Definition 3.2 (EvidenceUpdate).

A candidate evidence update e is a source-addressed package with declared source, type, scope, contract, evidence class, and provenance. Admission is a separately recorded gate

$$e \xrightarrow{\text{Admit}(\cdot; E_t, \Pi_{\text{adm},t})} \hat{e},$$

where the source-addressed result is recorded by r_{adm} and $\hat{e}$ exists only when that result is **ADMITTED**.

The complete set of possible admission results is

$$\{\text{ADMITTED}, \text{REJECTED}, \text{UNRESOLVED}\}.$$

Rejected or unresolved input may remain in an intake log, but it does not enter normative revision as admitted evidence. The admission rule is supplied by the active $\Pi_{\text{adm},t}$; it is not selected ad hoc by the proposed revision.

3.4 Dependency Relation

Definition 3.3 (Declared Dependency).

For each $c \in K_t$, the declared dependency pair consists of two finite typed components:

$$\text{Dep}_t^H(c) \subseteq H_t, \quad \text{Dep}_t^K(c) \subseteq K_t.$$

The first contains immutable history premises; the second contains current-use commitment entries on which the current status of c depends. Together they include the declared validation basis of c without conflating history references and commitment entries.

For commitment entries, write $d \prec_t c$ when $d \in \text{Dep}_t^K(c)$, and let $\prec_t^+$ be its finite transitive closure. For $S \subseteq K_t$, define the affected current entries

$$\text{Affected}_t(S) = \{c \in K_t \setminus S : \exists d \in S \text{ with } d \prec_t^+ c\}.$$

This reachability relation is defined only over declared normative dependencies. Undeclared operational dependencies lie outside the proposition.

4 Six System Principles

4.1 Prior Fallibility

Principle 1 (Prior Fallibility). A current realization, alignment, profile, policy, interpretation, or model reference is defeasible. Current status alone does not make it ground truth.

This principle is compatible with exact mathematics; it rules out the illicit promotion of a currently admitted application model to an unrevisable description of the object.

4.2 Action–Observation Separation

Principle 2 (Action–Observation Separation). An action changes an object only through a separately typed external transition. Observation produces evidence. When an external action is also intended as a probe, its intervention role and the evidential role of the subsequent observation must be recorded separately.

Thus

$$u_t : X_t \rightarrow X_{t+1} \quad \text{and} \quad \text{Observe}_{\eta, t+1} : X_{t+1} \rightarrow \mathcal{F}_{t+1}$$

are different arrows. Labeling u_t a probe does not merge these distinct transitions.

4.3 Evidence-Driven Revision

Principle 3 (Evidence-Driven Revision). Only evidence admitted under a declared source, type, scope, contract, and evidence policy may trigger a normative revision.

Let

$$e \xrightarrow{\text{Admit}(\cdot; E_t, \Pi_{\text{adm}, t})} \hat{e}.$$

Only $\hat{e}$ enters the revision map. Recency alone supplies no ordering between evidence packages. Newly admitted, scope-compatible evidence may supersede only commitments whose premises or declared revision conditions it affects under the active $\Pi_{\text{rev}, t}$.

4.4 Correction as a First-Class Transition

Principle 4 (Correction as a First-Class Transition). Correction is a typed transition between epistemic states, not deletion of an earlier state and not exceptional recovery from a malformed history. Epistemic correction is not object repair and is not intervention.

The completed transition is

$$E_t \xrightarrow{\mathcal{L}_{t \rightarrow t+1}} E_{t+1}.$$

Here $\mathcal{L}_{t \rightarrow t+1}$ denotes the full $\text{ProposeRevision} \rightarrow \text{ValidateRevision} \rightarrow \text{ActivateCurrent}$ lifecycle. The record $\rho_{t \rightarrow t+1}$ describes only the proposal and does not itself make E_{t+1} current.

It may occur while X_t remains fixed. For example, correcting a sector alignment can supersede an audit without changing either source snapshot.

epistemic correction $\neq$ object repair $\neq$ intervention.

4.5 Epistemic Revision Is Not Operational Rollback

Let $K \cong_K K'$ denote a declared commitment-level equivalence of current-use bindings. It is distinct from any operational or observational equivalence used by a runtime. Epistemic revision is not operational rollback. Whereas operational recovery may attempt to restore a prior runtime state, epistemic correction forms a successor current-use binding while preserving the admitted revision history. Reinstating an earlier position creates a successor epistemic state; it does not erase the intervening revision.

$$E_t \xrightarrow{\mathcal{L}_{t \rightarrow t+1}} E_{t+1}, \quad H_t \subseteq H_{t+1}.$$

Even if a later revision yields

$$E_{t+1} \xrightarrow{\mathcal{L}_{t+1 \rightarrow t+2}} E_{t+2}, \quad K_{t+2} \cong_K K_t,$$

the admitted history still satisfies

$$H_t \subseteq H_{t+1} \subseteq H_{t+2}.$$

Consequently,

$$K_{t+2} \cong_K K_t \not\Rightarrow E_{t+2} = E_t.$$

reinstatement $\neq$ rollback $\neq$ historical erasure.

4.6 Feedback Non-Authority

Principle 5 (Feedback Non-Authority). Feedback, challenge, negative assessment, or revision-required status does not select, authorize, or execute an action.

In particular, feedback does not imply authorization, and authorization does not imply execution.

New evidence may produce `REVIEW_REQUIRED`, a shadow revision, or a new current epistemic state. It cannot directly produce `EXECUTE`. The normative informational chain is

$$\text{evidence} \longrightarrow \text{admission} \longrightarrow \text{proposal} \longrightarrow \text{validation} \longrightarrow \text{activation}.$$

The activated state may later inform a separately scoped Paper XIV interpretation, but that invocation is not part of the revision transition.

4.7 Receding-Horizon Verifiability

Principle 6 (Receding-Horizon Verifiability). The next admitted epistemic transition should be justified by the currently admitted closure, replayable under its declared revision policy, and recorded so that its downstream commitments can later be revalidated.

The principle does not require the system to solve the whole future. It requires the next admitted transition to be locally justified and later auditable. It neither chooses an external action nor proves long-horizon optimality.

5 Typed Revision Interface

5.1 Revision Policy

Definition 5.1 (Revision Policy).

A revision policy $\Pi_{\text{rev},t}$ is a versioned finite rule object that references the active admission policy $\Pi_{\text{adm},t}$ and declares applicable trigger kinds, revision scopes, supersession conditions, retention conditions, activation conditions, and revalidation rules.

The policy is a normative input profile, not a sixth record/state object and not ground truth. Two admissible revision policies may produce different current commitments from the same history. Any relativity claim must therefore retain the policy reference. However, a revision cannot select an ad hoc policy: $\Pi_{\text{rev},t}$ is a versioned policy subject referenced by Γ_t and supported by a current commitment entry in K_t . Revision rules are themselves revisable commitments, and any successor policy must be installed through the previously active revision interface.

5.2 Supersession and Revalidation

Definition 5.2 (SupersessionMap).

Let $S \subseteq K_t$ be the superseded commitment entries, and let N be a finite set of proposed successor commitment entries, including any successors named by supersession. Every member of N has a fresh commitment-entry identity, so $N \cap K_t = \emptyset$. Despite its name, an explicit **SupersessionMap** is a finite typed relation

$$\Sigma_{t \rightarrow t+1} \subseteq S \times (N \sqcup \{\text{WITHDRAWN}\}),$$

with domain S . A relation edge names a successor entry or records withdrawal without replacement. The disjoint union keeps the distinguished withdrawal marker outside the successor-entry type. Multiple targets permit a split; multiple sources may share one target to record a merge. The relation does not force one-to-one succession and never rewrites a subject artifact.

Definition 5.3 (RevalidationObligation).

Let

$$Q_{t \rightarrow t+1} = \text{Affected}_t(S).$$

The finite object $V_{t \rightarrow t+1}$ records every member of $Q_{t \rightarrow t+1}$, the superseded dependency or dependencies that make it reachable, and the contract closure under which revalidation or regeneration must occur.

For a closed revision scope, the retained set and old-state partition are as follows; after validation and activation, the final line gives the new current set:

$$\begin{aligned} R &= K_t \setminus (S \cup Q_{t \rightarrow t+1}), \\ K_t &= S \dot{\cup} R \dot{\cup} Q_{t \rightarrow t+1}, \\ K_{t+1} &= R \cup N. \end{aligned}$$

A dependent named in $V_{t \rightarrow t+1}$ may return to a later current set only after its obligation is discharged by a new source-addressed validation or regeneration. The returning element is a successor commitment entry, even when it references the same immutable subject as the old entry. The initial lifecycle vocabulary is:

Term	Revision meaning
CURRENT	eligible for current use under the active epistemic state
SUPERSEDED	preserved historically but no longer current
REVALIDATION_REQUIRED	current validity is not inherited after dependency change
RETAINED	remains current through this revision
WITHDRAWN	superseded without a named successor

RETAINED is a revision-local disposition: the commitment entry continues as CURRENT after the revision. SUPERSEDED applies to a current-use binding; it does not mean that its subject artifact is FALSE, INVALID_AT_CREATION, or deleted. A subject may become current again only through a later explicit commitment entry. Reinstatement does not erase the earlier supersession edge.

5.3 Revision Map and Record

For admitted $\hat{e}$, write the partial typed proposal map

$$\text{ProposeRevision} : (E_t, \hat{e}, \Pi_{\text{rev},t}) \rightarrow (\tilde{E}_{t+1}, \rho_{t \rightarrow t+1}, \Sigma_{t \rightarrow t+1}, V_{t \rightarrow t+1}).$$

The map is partial because admitted evidence may still fall outside the policy scope or leave the proposed revision unresolved. No default correction is fabricated. The proposed state $\tilde{E}_{t+1}$ is not current.

Definition 5.4 (RevisionRecord).

A revision record has the semantic form

$$\begin{aligned} \rho_{t \rightarrow t+1} = & (\text{id}, E_t^{\text{ref}}, \hat{e}^{\text{ref}}, r_{\text{adm}}^{\text{ref}}, \\ & \Pi_{\text{adm},t}^{\text{ref}}, \Pi_{\text{rev},t}^{\text{ref}}, \\ & \text{trigger}, \text{scope}, S, R, N, \\ & \Sigma_{t \rightarrow t+1}^{\text{ref}}, V_{t \rightarrow t+1}^{\text{ref}}, \text{reason code}, \\ & \text{causal boundary}, \text{provenance closure}). \end{aligned}$$

The record contains neither the conformance receipt that validates it nor a reference to the final current state. It therefore cannot depend on its own validation or on a downstream state whose history includes the record. The proposed state is derived from E_t , $\rho_{t \rightarrow t+1}$, Σ , and V ; a separate conformance receipt may then bind the exact proposal.

Every source entry of $\Sigma_{t \rightarrow t+1}$ remains represented in H_{t+1} together with its immutable subject reference and original declared semantic binding. A member of R remains current because it passed the revision's dependency check. A member of N must be newly bound to the admitted evidence, validation basis, and revision-policy closure proposed for its current use. The separate activation basis determines whether the proposal becomes current. Its `subject_ref` may identify either a new artifact or the same immutable artifact referenced by a superseded entry.

The trigger, scope, and reason code are typed descriptive metadata. They record why an epistemic revision was formed and which informational commitments it addresses; they do not identify what caused an object-state transition.

5.4 Validation and Activation

Every revision is first evaluated without making its result current:

$$\text{ValidateRevision}(\widetilde{E}_{t+1}, \rho_{t \rightarrow t+1}) = r_{\text{conf}}.$$

$\widetilde{E}_{t+1}$ is non-authoritative. It can be validated, compared, or discarded while E_t remains current. Conformance validation alone cannot grant currentness. Activation requires both an explicit activation basis b_{act} and a conformance receipt r_{conf} . When the declared activation predicates hold, the partial operation is defined by

$$\text{ActivateCurrent}(\widetilde{E}_{t+1}, \rho_{t \rightarrow t+1}, b_{\text{act}}, r_{\text{conf}}) = E_{t+1}.$$

The activation basis may cite an automatic-promotion condition already allowed by the active $\Pi_{\text{rev},t}$ or a separately supplied governance or review reference. This paper does not define who may issue that external basis; it requires the basis to be explicit and source-addressed. The conformance receipt establishes local protocol conformance only and is downstream of the proposal it validates. The activated history contains references to the admitted evidence, admission receipt, proposal record, supersession relation, revalidation obligation, conformance receipt, activation basis, and new commitment entries; none is rewritten to point back to the activated state. Therefore

$$r_{\text{conf}} = \text{VALID} \not\Rightarrow \text{currentness authorized}.$$

Activation is an epistemic-state transition, not object execution and not a claim that the new state is ground truth. An activation basis grants at most the declared epistemic-currentness role; it is not candidate-specific action authorization and cannot be substituted for one.

An unactivated proposal serves as this paper's shadow-revision construction. No separate direct path exists to make a proposal current before validation.

5.5 Authority Boundary

A revision record is an informational object. It contains no object-state transition, selection, authorization, execution, outcome, effect, or causal semantics. Any serialized representation must preserve these exclusions.

Generic evidence or provenance references may not be used to smuggle a repair command, authorization receipt, executor result, post-action outcome, or effect certificate into the revision artifact.

A later revision may cite a separately validated external artifact as an evidence reference. It may not embed, reclassify, or validate that artifact as part of the revision interface. Merely referencing an execution result does not turn the revision record into an execution receipt.

6 Interface Propositions

6.1 Provenance-Monotone, Commitment-Nonmonotone Principle

Proposition 6.1 (Provenance-Monotone, Commitment-Nonmonotone).

For every conforming revision,

$$H_t \subseteq H_{t+1},$$

while neither $K_t \subseteq K_{t+1}$ nor $K_{t+1} \subseteq K_t$ is required.

Proof. An activated conforming revision appends references to the admitted evidence, admission receipt, revision record, supersession relation, revalidation obligation, conformance receipt, activation basis, and new commitment entries to H_t and removes no prior history reference. Hence $H_t \subseteq H_{t+1}$. By the typed revision construction, $K_{t+1} = R \cup N$ with the source entries of $\Sigma_{t \rightarrow t+1}$ removed from current status and N possibly new. The successor entries may reference new subjects or reuse old immutable subjects under new validation bases. If superseded and new entry sets are both nonempty, neither inclusion between the commitment-entry sets follows.

This is the central separation: **history is append-only; current commitment is corrigible.** $\square$

6.2 Non-Retroactive Semantic-Binding Principle

Proposition 6.2 (Non-Retroactive Semantic Binding).

Let $c \in K_t$ be a commitment entry with immutable subject $a = \text{subject_ref}(c)$. If a conforming revision supersedes c , it preserves the bytes, identity, and declared historical semantic binding of a . Every successor c' is a distinct commitment entry. It may reference the same subject a under a new scope or validation basis, or a distinct corrected artifact a' . Neither case retroactively rebinds the old bytes to successor semantics.

Proof. By the `EpistemicState` definition, a denotes one immutable payload together with the declared contract and validation closure under which that payload was formed. Currentness instead belongs to the commitment entry c . Replacing the payload or rebinding it to a successor contract would violate the historical binding. A validation-only revision can therefore form a distinct entry c' with the same `subject_ref` and a new `validation_basis_ref`; a content correction must form both a new artifact identity and a new entry. The `SupersessionMap` relates the old and new entries without rewriting either subject.

A prior validation receipt remains an immutable record of what its validator returned under the declared historical closure. It does not guarantee that the old binding remains current, or prevent a later interpretation from superseding it. Historical semantic binding and current applicability are different predicates. In particular,

$$\text{SUPERSEDED} \neq \text{FALSE} \neq \text{INVALID_AT_CREATION}.$$

$\square$

6.3 Revalidation Obligation

Proposition 6.3 (Revalidation Obligation).

Let $c, d \in K_t$ with $d \prec_t^+ c$. If a conforming revision supersedes the dependency binding d , then the old entry c cannot remain in the retained set R . If c is itself superseded, then $c \in S$; otherwise $c \in Q_{t \rightarrow t+1}$ and is named by $V_{t \rightarrow t+1}$. Current use may be restored only by a fresh successor entry $c' \in N$ whose validation basis is bound to an admissible dependency closure under E_{t+1} , even when $\text{subject_ref}(c') = \text{subject_ref}(c)$.

Proof. The `validation_basis_ref` of c binds a declared dependency path to the current-use entry d , not to a successor binding. Non-Retroactive Semantic Binding forbids silently treating those bindings as identical. By Definition 5.3, every affected entry outside S belongs to $Q_{t \rightarrow t+1}$,

and $R = K_t \setminus (S \cup Q_{t \rightarrow t+1})$. Therefore c is not retained. Restoring current use requires a fresh successor entry with a new validation binding; absent one, the revalidation obligation remains outstanding.

This proposition states a dependency-scoped obligation, not an algorithm for computing or scheduling revalidation. $\square$

6.4 Typed Non-Authority Result

Proposition 6.4 (Feedback Non-Authority).

An admitted evidence update, challenge, RevisionRecord, SupersessionMap, or RevalidationObligation does not by itself authorize or execute an object-state transition.

Proof. The revision and activation codomains contain only informational objects $(\tilde{E}_{t+1}, \rho, \Sigma, V)$ and E_{t+1} . Their fixed negative boundary has no selected-plan, authorization, executor, outcome, or effect semantics. Therefore no term of the revision interface inhabits an authorization or object-transition type. Such an arrow remains outside this paper and requires an external, system-specific interface. $\square$

6.5 No Convergence Without Additional Structure

Proposition 6.5 (No Convergence Without Additional Structure).

Typed, provenance-preserving, and replayable revision does not by itself imply that the current epistemic projection (K_t, Γ_t) stabilizes or converges.

Proof by construction. Let the possible current commitment sets be $K^{(0)} = \{k_0\}$ and $K^{(1)} = \{k_1\}$. Let admitted evidence packages $e_0, e_1, e_0, e_1, \dots$ alternate. Choose a versioned policy that retains $K^{(i)}$ exactly when the latest admitted package is e_i , superseding the other commitment and appending a conforming revision record. Then H_t grows monotonically, every step is typed and replayable, but $K_t = K^{(0)}, K^{(1)}, K^{(0)}, K^{(1)}, \dots$ never stabilizes.

Convergence requires additional structure such as a finite terminating policy, a contraction, a monotone objective, bounded variation, stable evidence, or another explicit hypothesis. None is supplied by the revision type alone.

7 Receding-Horizon Revision Architecture

7.1 Closed Informational Loop

This paper consumes admitted evidence, regardless of whether that evidence followed passive observation, an external intervention, or a correction to an existing informational artifact. Its complete owned path is

$$\begin{aligned}
 e_{t+1} &\xrightarrow{\text{Admit}(\cdot; E_t, \Pi_{\text{adm}, t})} \hat{e}_{t+1}, \\
 (E_t, \hat{e}_{t+1}, \Pi_{\text{rev}, t}) &\xrightarrow{\text{ProposeRevision}} (\tilde{E}_{t+1}, \rho, \Sigma, V), \\
 (\tilde{E}_{t+1}, \rho) &\xrightarrow{\text{ValidateRevision}} r_{\text{conf}}, \\
 (\tilde{E}_{t+1}, \rho, b_{\text{act}}, r_{\text{conf}}) &\xrightarrow[\text{when defined}]{\text{ActivateCurrent}} E_{t+1}.
 \end{aligned}$$

Any object transition $X_t \rightarrow X_{t+1}$ and any selection, planning, authorization, execution, outcome, or effect semantics remain external. They are not unfinished SOF protocol stages. An epistemic state has no object-layer agency, and the source of an evidence package does not change the type of the revision interface.

This paper closes the numbered SOF protocol line. Action execution does not become a successor protocol stage. Subsequent contextual and descent mathematics belongs to an independently scoped mathematical line.

7.2 Local Admission Rather Than Global Precomputation

A revision step is locally justified when:

1. its new evidence is admitted under the current source and evidence policy;
2. its active admission, revision, and contract references are explicit;
3. its superseded, retained, new, and revalidation-required references are closed and replayable;
4. its proposal, revision record, validation receipt, and activation basis are source-addressed and ordered without a forward or cyclic validation dependency;
5. it makes no undeclared object-transition or causal claim.

These conditions support auditing the next epistemic step. They do not prove that the chosen external action is optimal, that the future evidence will be informative, or that a finite sequence of local revisions reaches a globally best model.

8 Boundary Examples

8.1 Alignment Correction with Fixed Source

Suppose two immutable reports are compared under alignment Φ_t and yield audit $\mathcal{A}_t$. New admitted evidence shows that Φ_t mismatched two sector labels. A revision may supersede the current-use entries whose subjects are Φ_t and $\mathcal{A}_t$, retain the entries for the two reports, introduce a new alignment artifact Φ_{t+1} with a new commitment entry, and require regeneration of the audit. The object snapshots, report bytes, and historical binding of Φ_t remain fixed. This is epistemic correction without object intervention.

8.2 Object Change Without Immediate Revision

Suppose an external process changes X_t to X_{t+1} . Until a new observation is admitted, E_t need not change. The world may have changed while the current epistemic state is stale. The revision interface does not infer the missing observation.

8.3 Inadmissible New Information

Suppose a newer payload lacks source provenance or violates the declared carrier contract. Recency does not authorize revision. Admission returns REJECTED or UNRESOLVED, and no normative E_{t+1} is formed from that payload.

8.4 Mismatch Without Repair

An audit mismatch may trigger revision of a realization, evidence reference, alignment, profile, policy, or interpretation. It does not imply that the object is defective or that an intervention candidate should be executed.

8.5 Probe Without Semantic Collapse

An external operator may execute an action partly to obtain information. An externally supplied execution record, if available, establishes only its declared execution event. The later observation and report establish the evidence consumed by revision. One artifact cannot silently serve both roles.

8.6 Contestability Without Adjudication

A generic challenge may be written

$$\text{Challenge}(a, r, e),$$

where a is the challenged artifact, r is a declared reason, and e is an evidence reference submitted for admission. A challenge is an intake event, not a sixth core artifact and not proof that a is false. After admission and review, the typed outcomes are:

- RETAINED;
- REVALIDATION_REQUIRED;
- SHADOW_REVISION;
- REVISED;
- UNRESOLVED.

Who may challenge, who must review, which appeal or remedy is available, and which rights constrain the process belong to an external governance specialization. This paper supplies only the generic corrigibility interface.

9 Relation to Papers IX–XIV

Paper	Owned object	Use in this paper	Non-ownership
IX	ObjectTrajectory, SOF0bservationRecord, DeformationRecord	may consume an admitted observation/deformation record as evidence	does not generate the object trajectory or identify its mechanism
X	capability declarations, typed IR, compiler profile, compiler output	may reference capability/evidence premises and their validation closure	does not alter compiler ownership or compiled claim identity
XI	wall morphology records and derived signatures	may supersede or retain a wall-record application reference	does not redefine wall admission or taxonomy
XII	immutable realization-relative SOF Report	may retain, supersede, or request revalidation of report-dependent commitments	does not mutate a report or claim report-level intervention semantics
XIII	explicit alignment and factual SOF Audit	may revise alignment/profile references and supersede dependent audits	does not promote mismatch to defect or causal attribution
XIV	context/policy interpretation and bounded candidates	may retain or supersede interpreta- tion/policy/candidate references	does not select, authorize, execute, or certify an effect

Thus Papers IX–XIV produce or consume typed informational artifacts, this paper records how their current-use closure is revised, and external systems alone own object intervention.

10 Claim Status and Boundary

Definitions and design principles are not independent evidence claims. The reader-facing claim spine is:

Object or conclusion	Treatment	Reader-facing status
EpistemicState, EvidenceUpdate, RevisionRecord, SupersessionMap, RevalidationObligation	five owned type definitions	not an independent evidence claim
AdmissionPolicy, RevisionPolicy	active versioned normative input profiles	not independent evidence claims
six system principles	normative interface position	not an independent evidence claim
provenance-monotone, commitment-nonmonotone separation	interface proposition under the declared revision construction	Theorem
non-retroactive semantic binding	interface proposition separating immutable artifact identity from current-use binding	Theorem
revalidation obligation	interface proposition under declared dependencies	Theorem
feedback non-authority	type-level proposition separating revision from authorization and object intervention	Theorem
oscillatory revision construction	counterexample to unconditional current-state stabilization	Theorem
shadow revision and contestability	non-authoritative typed constructions	not an independent evidence claim
finite boundary examples	typed illustrations without project-specific data	not a computational claim
convergence under additional structural hypotheses	open problem	Research Program

This paper makes no Computational Certificate or Computational Observation claim. It does not claim:

1. that an epistemic state equals or converges to the object state;
2. that newer evidence dominates older evidence without admission and scope;
3. that a revision reason identifies a cause of object change;
4. that audit mismatch implies object defect or intervention;
5. that a correction mutates a prior validated artifact;
6. that revalidation guarantees a favorable conclusion;
7. that typed revision converges, terminates, or reaches a global optimum;
8. that this paper selects, authorizes, executes, or evaluates an action;
9. that a `RevisionRecord` is an execution or effect receipt.

11 Conclusion

The SOF stack requires a type for changing its mind without rewriting its past. This paper formalizes an acyclic epistemic transition:

$$\text{ProposeRevision} \longrightarrow \text{ValidateRevision} \longrightarrow \text{ActivateCurrent}.$$

The object state and epistemic state remain distinct. Provenance grows while current-use commitment entries may be withdrawn. Supersession creates new bindings and explicit semantic edges while preserving each artifact’s bytes and declared historical semantic binding. A new binding may reuse the same immutable artifact under a new validation basis. A dependent conclusion does not inherit current validity after its premise changes. Feedback has no action authority, and conformance alone has no currentness authority. Each revision can be locally typed, replayable, and auditable without any promise that current commitments stabilize.

The resulting program philosophy is modest but operationally strong: SOF does not assume that sufficient precomputation eliminates uncertainty. It makes observation, correction, provenance-preserving revision, and revalidation first-class components of a composable structural interface.

Papers VIII–XIV make the interface composable and verifiable. This paper adds corrigibility and closes the numbered protocol line.

12 Outlook

The next mathematical questions are conditional rather than universal.

1. Identify explicit hypotheses under which a revision sequence terminates or converges, including finite hypothesis classes, contractions, monotone objectives, bounded variation, or stable evidence processes.
2. Define equivalence and refinement relations between epistemic states without collapsing distinct histories.
3. Study dependency-scoped revalidation semantics and sufficient conditions for preserving unaffected current commitments.
4. Develop an independent contextual/descent mathematical line for aligned typed realizations, restriction maps, finite descent controls, and declared composition signatures. Such work receives its own scope outside this protocol sequence.

References

- [1] Carlos E. Alchourrón, Peter Gärdenfors, and David Makinson. On the logic of theory change: Partial meet contraction and revision functions. *The Journal of Symbolic Logic*, 50(2):510–530, 1985.
- [2] Jay Banerjee, Won Kim, Hyoung-Joo Kim, and Henry F. Korth. Semantics and implementation of schema evolution in object-oriented databases. In *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, pages 311–322, 1987.
- [3] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and where: A characterization of data provenance. In *Database Theory – ICDT 2001*, volume 1973 of *Lecture Notes in Computer Science*, pages 316–330. Springer, 2001.

- [4] WuJun Chen. Capability-aware compilation for sectorized observable frameworks: Typed admission and cross-species registry evidence. *Preprint*, 2026. Paper X of the RIME program, version 2.0. DOI: <https://doi.org/10.5281/zenodo.21768257>.
- [5] WuJun Chen. Observable dynamics of sectorized observable frameworks: Typed deformations, observable trajectories, and wall pullbacks. *Preprint*, 2026. Paper IX of the RIME program, version 2.1. DOI: <https://doi.org/10.5281/zenodo.21991080>.
- [6] WuJun Chen. Sectorized observable framework: A typed static object language for sectorized observables. *Preprint*, 2026. Paper VIII of the RIME program, version 2.1. DOI: <https://doi.org/10.5281/zenodo.21977464>.
- [7] WuJun Chen. SOF action semantics: Contextual interpretation and bounded candidate dispositions. *Preprint*, 2026. Paper XIV of the RIME program, version 2.1. DOI: <https://doi.org/10.5281/zenodo.22024158>.
- [8] WuJun Chen. SOF diagnostic protocol: Realization-relative reports for strict SOF realizations and diagnostic analogues. *Preprint*, 2026. Paper XII of the RIME program, version 2.1. DOI: <https://doi.org/10.5281/zenodo.22006816>.
- [9] WuJun Chen. Typed alignment and audit signatures: Realization-relative comparison for SOF reports. *Preprint*, 2026. Paper XIII of the RIME program, version 2.1. DOI: <https://doi.org/10.5281/zenodo.22010220>.
- [10] WuJun Chen. Typed wall morphology for sectorized observable frameworks: Sparse wall records, coordinate profiles, multi-label taxonomy, and local-model eligibility. *Preprint*, 2026. Paper XI of the RIME program, version 2.0. DOI: <https://doi.org/10.5281/zenodo.21801722>.
- [11] Jon Doyle. A truth maintenance system. *Artificial Intelligence*, 12(3):231–272, 1979.
- [12] David Q. Mayne, James B. Rawlings, Christopher V. Rao, and Pierre O. M. Scokaert. Constrained model predictive control: Stability and optimality. *Automatica*, 36(6):789–814, 2000.
- [13] Yifan Shi, Wei Zhang, and Tianyi Cui. A programming paradigm for spatiotemporal composability, 2026. Preprint, draft of 13 August 2026; repository revision 948a07b.
- [14] Yogesh L. Simmhan, Beth Plale, and Dennis Gannon. A survey of data provenance in e-science. *ACM SIGMOD Record*, 34(3):31–36, 2005.